

Sheet 5.

1. a)

$$D_{KL}[Q(z|X) \parallel P(z|X)] = \sum_z Q(z|X) \log \frac{Q(z|X)}{P(z|X)}$$

$$= \sum_z Q(z|X) [\log Q(z|X) - \log P(z|X)]$$

$$= E_{z \sim Q(z|X)} [\log Q(z|X) - \log P(z|X)]$$

$$\stackrel{\text{Bayes}}{=} E_{z \sim Q(z|X)} [\log Q(z|X) - \log P(X|z) - \log P(z) + \log P(X)]$$

$$= E_{z \sim Q(z|X)} [\log Q(z|X) - \log P(X|z) - \log P(z) + \log P(X)]$$

$$P(z|X) = \frac{P(X|z)P(z)}{P(X)}$$

1. b)

$$\log P(X) - D_{KL}[Q(z|X) \parallel P(z|X)] =$$

$$= -E_{z \sim Q(z|X)} [\log Q(z|X) - \log P(X|z) - \log P(z)]$$

$$= E_{z \sim Q(z|X)} [P(X|z)]$$

$$+ E_{z \sim Q(z|X)} [\log P(z) - \log Q(z|X)]$$

$$= E_{z \sim Q(z|X)} [P(X|z)] - D_{KL}[Q(z|X) \parallel P(z)]$$

In this task we implement a very simple version of GPT ourselves.

Grammar Description

This grammar generates sentences where a subject (a cat, dog, or bird) performs an action (jumps, runs, or flies) and either goes over another noun or goes through the air.

The grammar rules are:

- `<start> ::= <sentence>`
- `<sentence> ::= <subject> <verb> <object> '.'`
- `<subject> ::= 'the' <noun>`
- `<noun> ::= 'cat' | 'dog' | 'bird'`
- `<verb> ::= 'jumps' | 'runs' | 'flies'`
- `<object> ::= 'over' 'the' <noun> | 'through' 'the' 'air'`

Example Sentences

- The dog runs over the cat.
- The bird flies through the air.
- The cat jumps over the bird.
- The dog runs through the air.
- The bird flies over the dog.

```
In [5]: import random

def generate_sentence():
    sentence = generate_subject() + generate_verb() + generate_object() +
    return sentence

def generate_subject():
    return 'the ' + random.choice(['cat', 'dog', 'bird']) + ' '

def generate_verb():
    return random.choice(['jumps', 'runs', 'flies']) + ' '

def generate_object():
    if random.random() < 0.5:
        return 'over the ' + random.choice(['cat', 'dog', 'bird']) + ' '
    else:
        return 'through the air '

# Generate and save 100 sentences to a text file
with open('sentences.txt', 'w') as f:
    for i in range(100):
        sentence = generate_sentence()
        f.write(sentence + '\n')
        if i % 10 == 0:
            print(sentence)
```

```

the dog runs over the dog .
the bird jumps through the air .
the cat flies over the dog .
the dog jumps over the cat .
the bird jumps over the cat .
the dog jumps through the air .
the bird jumps through the air .
the cat runs through the air .
the bird jumps through the air .
the cat jumps over the dog .

```

Next we create a dataloader loading the created dataset:

```

In [6]: import torch
        from torch.utils.data import Dataset, DataLoader

        class SentenceDataset(Dataset):
            def __init__(self, filename, tokenizer, max_len=10):
                with open(filename, 'r') as f:
                    self.sentences = [line.strip() for line in f.readlines()]

                self.tokenizer = tokenizer
                self.max_len = max_len

            def __len__(self):
                return len(self.sentences)

            def __getitem__(self, idx):
                sentence = '<S> ' + self.sentences[idx] + ' <EOS>'
                words = sentence.split()
                words = words[:self.max_len] + ['<EOS>'] * max(0, self.max_len -
                return torch.tensor([self.tokenizer[word] for word in words])

```

Task 1 (10 P)

Implement the mask in the forward pass to avoid looking at future tokens during training

```

In [7]: import torch
        import math
        import torch.nn as nn
        import torch.nn.functional as F

        class GPT(nn.Module):
            def __init__(self, vocab_size, embedding_dim, num_heads, num_layers):
                super().__init__()

                # Embedding layer
                self.embedding = nn.Embedding(vocab_size, embedding_dim)

                # Multi-head attention layers
                self.num_heads = num_heads
                self.attention_layers = nn.ModuleList([
                    nn.MultiheadAttention(embed_dim=embedding_dim, num_heads=num_
                    for _ in range(num_layers)
                ])

```

```

self.layer_norms1 = nn.ModuleList([
    torch.nn.LayerNorm(embedding_dim)
    for _ in range(num_layers)
])

# Position-wise feedforward layers
self.feedforward_layers = nn.ModuleList([
    nn.Sequential(
        nn.Linear(embedding_dim, embedding_dim),
        nn.ReLU(),
    )
    for _ in range(num_layers)
])

self.layer_norms2 = nn.ModuleList([
    torch.nn.LayerNorm(embedding_dim)
    for _ in range(num_layers)
])

self.unembed = nn.Linear(embedding_dim, vocab_size)

def forward(self, input_ids):
    # input_ids.shape = (N, 10)

    # Embed input sequence
    x = self.embedding(input_ids)
    #print(f"{x.shape=}")
    # x.shape = (N, embedding_dim)

    # Calculate self-attention for each layer
    for attention_layer, feedforward_layer, layer_norm1, layer_norm2:
        pos_enc = torch.zeros(x.shape[1], x.shape[-1])
        # calculate the position and dimension values for each element
        pos = torch.arange(x.shape[1], dtype=torch.float).unsqueeze(1)
        div = torch.exp(
            torch.arange(0, x.shape[-1], 2).float() *
            (-math.log(10000.0) / x.shape[-1])
        )
        # apply the sin/cos formula to each element in the matrix
        pos_enc[:, 0::2] = torch.sin(pos * div)
        pos_enc[:, 1::2] = torch.cos(pos * div)
        pos_enc = pos_enc.unsqueeze(0)

        x = x + pos_enc.to(x)

        # TODO create a mask to prevent the model from attending to f
        mask = torch.triu(
            torch.full((x.shape[1], x.shape[1]), float('-inf')),
            diagonal=1,
        )
        attn_output, _ = attention_layer(x, x, x, attn_mask=mask)
        # END

        attn_output = attn_output + x
        attn_output = layer_norm1(attn_output)

    # Position-wise feedforward layer
    ff_output = feedforward_layer(attn_output)
    ff_output = ff_output + attn_output
    ff_output = layer_norm2(ff_output)

```

```

        # Update input embeddings for next layer
        x = ff_output
        #print(f"{x.shape=}")
        return self.unembed(x)

```

Task 2 (10 P)

Implement and run the training loop

```

In [8]: import torch.optim as optim

# Define training hyperparameters
learning_rate = 0.001
batch_size = 32
num_epochs = 10

# Define tokenizer and GPT model
tokenizer = {"<S>": 0, "<EOS>": 1, "the": 2, "dog": 3, "cat": 4, "bird": 5}
gpt_model = GPT(vocab_size=len(tokenizer), embedding_dim=64, num_heads=4,
                 num_layers=6)

# Define loss function and optimizer
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(gpt_model.parameters(), lr=learning_rate)

# Load training data
training_data = SentenceDataset("sentences.txt", tokenizer)

# Define dataloader
dataloader = torch.utils.data.DataLoader(training_data, batch_size=batch_size,
                                          shuffle=True)

# Train model
for epoch in range(num_epochs):
    running_loss = 0.0
    for i, batch in enumerate(dataloader):
        # batch.shape = (batch_size, 10)

        # Zero gradients
        optimizer.zero_grad()

        # Forward pass
        # TODO create input_ids and target_ids based on the current batch

        input_ids = batch[:, :-1]
        target_ids = batch[:, 1:]

        outputs = gpt_model(input_ids)
        # Compute loss and backpropagate
        loss = criterion(outputs.permute(0, 2, 1), target_ids)
        loss.backward()
        optimizer.step()

        # Print loss statistics
        running_loss += loss.item()

    print(f"Epoch {epoch + 1}, Loss: {running_loss / 10}")
    running_loss = 0.0

```

```
Epoch 1, Loss: 1.0260919332504272
Epoch 2, Loss: 0.7839654207229614
Epoch 3, Loss: 0.6324350833892822
Epoch 4, Loss: 0.517425537109375
Epoch 5, Loss: 0.42689884901046754
Epoch 6, Loss: 0.3389808595180511
Epoch 7, Loss: 0.27749996781349184
Epoch 8, Loss: 0.23778040409088136
Epoch 9, Loss: 0.21533562541007994
Epoch 10, Loss: 0.18942081928253174
```

Task 3 (30 P)

Write a method for sampling a sentence from the model

```
In [9]: import random

def sample_sentence(model, tokenizer, max_len=10):
    # TODO implement sentence sampling for the given model that returns t
    tokenizer_inv = {v: k for (k, v) in tokenizer.items()} # int -> str
    output = torch.tensor([[tokenizer['<S>']]])
    with torch.no_grad():
        for _ in range(max_len):
            logits = model(output)

            # highest scoring token
            idx = int(torch.argmax(logits, dim=-1)[0, -1])

            # random topk token
            # topK = torch.topk(logits[0, -1].flatten(), k=3).indices
            # idx = int(random.choice(topK))

            token = tokenizer_inv[idx]
            if token == '<EOS>':
                break
            output = torch.cat([output, torch.tensor([[idx]])], dim=1)

    # print(output)
    return " ".join(map(lambda t: tokenizer_inv[int(t)], output[0, 1:]))

sample_sentence(gpt_model, tokenizer)
```

Out[9]: 'the dog runs through the air .'

Now we sample 10 sentences from GPT:

```
In [10]: def sample_sentences(model, tokenizer, num_sentences=10, max_len=10):
    # Generate num_sentences sentences
    sentences = []
    for i in range(num_sentences):
        sentence = sample_sentence(model, tokenizer, max_len=max_len)
        sentences.append(sentence)

    return sentences

# Generate 10 sample sentences
```

```
sampled_sentences = sample_sentences(gpt_model, tokenizer, num_sentences=  
  
for sentence in sampled_sentences:  
    print(sentence)
```

```
the dog runs through the air .  
the dog runs through the air .  
the dog runs through the air .  
the dog runs through the air .  
the dog runs through the air .  
the dog runs through the air .  
the dog runs through the air .  
the dog runs through the air .  
the dog runs through the air .  
the dog runs through the air .
```

In []: